WoSM Academy

CS102-F26 · DIGITAL DESIGN AND COMPUTER ARCHITECTURE I

Building and Programming the DX8

Lecture 22: putting it all together

Dr. Ali Khudiyev

Fall 2026

CC BY-SA 4.0

Today

01 The whole DX8

02 Programming the DX8

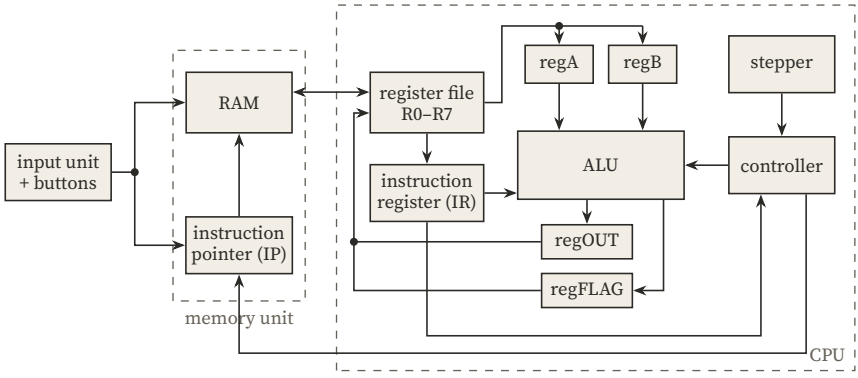
03 The project

04 Summary

01

The whole DX8

All the parts, together



- The book's Figure 6.13 connects everything of Units 15 to 21, with a few more registers to hold the instruction, the ALU's inputs and its results.

One instruction in eight steps

Step	Stage	In the book's circuit
s_1	fetch	the byte at address IP, $b_{15:8}$, is copied into R0
s_2	fetch	the byte at IP +1, $b_{7:0}$, is copied into R1
s_3	decode	R0 goes to the instruction register: b_{15}, opcode, register
s_4	decode	R1 goes to the argument register: `val` or the address
s_5	execute	the ALU's result goes to regOUT, its flags to regFLAG
s_6	execute	(nothing more to do)
s_7	load	results are written back: register file, regA, regB or RAM
s_8	load	the IP moves on to IP +2, or to `val` after a jump

- regOUT and regFLAG are loaded at step s_5 of *every* instruction, so they are only temporary. A result that must last is written to a register in step s_7.

The load stage, instruction by instruction

Instruction	In step s_7	In step s_8
ALU instruction	regOUT → the register in $b_{10:8}$	IP +2
`move`	`val` → the register in $b_{10:8}$	IP +2
`arg1`, `arg2`	the register in $b_{10:8}$ → regA, regB	IP +2
`store`	the register in $b_{10:8}$ → M[`memAddr`]	IP +2
`jmp`		`val`
`jmpgt`, …		`val` if the flag is set, else IP +2

- An ALU instruction does use its register field: it names the register that receives the result. `cmp` writes 0 there, since its answer is in the flags.
- Each instruction is 2 bytes long, so “the next instruction” (regIP +1 in Table 6.2) is 2 addresses further on.

Rules for DX8 programs

- **Two bytes per instruction**: $b_{15:8}$ at an even address, $b_{7:0}$ at the next one.
- **Start at 0**: reset sets the IP to 0 and the stepper to step 1.
- **Leave R0 and R1 alone**: every fetch writes the instruction into them. Keep your values in R2 to R7.
- **Choose where results go**: an ALU instruction writes its result into the register in $b_{10:8}$.
- **The flags compare regA with regB**, as they are at step s_5 of the instruction: a conditional jump tests the values put there by the last `arg1` and `arg2`.
- **There is no halt**: end with a jump to the jump itself.

02

Programming the DX8

Switches and buttons

Definition (Button)

A (push) *button* sends 1 as long as it is pressed; its output drops to 0 as soon as you lift your finger.

Definition (Switch)

A *switch* sends 1 once you turn it on, and keeps sending 1 until you turn it off again.

The DX8's "keyboard" `(Unit 15)`:

- 8 *data* switches and 8 *address* switches;
- a *write* button, which stores the data at the address;
- a *reset* switch.

Typing in a program

1. Turn on the reset switch.
2. Set the data switches to the next byte of the program.
3. Set the address switches to its address.
4. Push the write button (as if you turned a switch on and off at once).
5. Repeat 2 to 4 until every byte of the program and its data is in the RAM.
6. Turn off the reset switch: the DX8 starts running from address 0.

> **Remark**
>
> The program is just bytes in the RAM, typed in like data: the Von Neumann idea (Unit 15), at work.

A program: 3×4 by repeated addition

Addr.	Bytes	Instruction	
00	12 00	move R2, 0	(the product)
02	13 04	move R3, 4	
04	14 03	move R4, 3	(the counter)
06	15 FF	move R5, −1	
08	16 00	move R6, 0	
0A	1A 00	arg1 R2	← loop
0C	23 00	arg2 R3	
0E	8A 00	add → R2	(R2 ← R2 + 4)
10	1C 00	arg1 R4	
12	25 00	arg2 R5	

Addr.	Bytes	Instruction	
14	8C 00	add → R4	(R4 ← R4 − 1)
16	1C 00	arg1 R4	
18	26 00	arg2 R6	
1A	87 00	cmp → R7	(R4 vs 0)
1C	30 0A	jmpgt 0A	
1E	1A 00	arg1 R2	
20	26 00	arg2 R6	
22	8F 00	add → R7	(R2 + 0)
24	28 24	jmp 24	(stop)

Addresses and bytes in hexadecimal. −1 is FF in two's complement (Unit 16), so adding it counts down.

Running it

After	R2	R4	jump?
set-up	0	3	
pass 1	4	2	yes
pass 2	8	1	yes
pass 3	12	0	no

- Each pass adds 4 to R2 and takes 1 from R4. While $R4 > 0$, `jmpgt` goes back to address `0A`.
- At the end, regOUT shows 12 (`0000 1100`), and the DX8 keeps jumping to `24`.
- 38 instructions of 8 cycles each run before the stop: 304 clock cycles.

Remark

The DX8 has no multiplication instruction; loops of additions do the job. Real processors often have a multiplier circuit for it.

03

The project

The DX8 project

- The circuit-design project of this module: build the DX8 computer, or an extension of it, in Logisim-evolution, and run your own programs on it. It counts for 40% of the module.
- Build it from your own parts of Units 6 to 21, and test each one before you connect it.
- Ideas for an extension: a `sub` instruction in one of the 12 unused opcodes; a `halt`; an output display for regOUT; a `load` of a byte from the RAM into a register.

> **Remark**
>
> The book's epilogue admits that the DX8 was built without the setup and hold checks of Unit 14. Doing them for your DX8 is a good extension too.

Try it

1. In which step does the IP change, and by how much?
   In s_8: by 2, or to `val` after a jump.
2. Write `move` R5, `0x0A` as two bytes.
   `15 0A`: 0 0010 101, then 0000 1010.
3. Which one byte of the program would you change to compute 3×5?
   The byte at address `03`: `04` becomes `05`.
4. Why may a program not keep a value in R1?
   Every fetch writes the second byte of the instruction into R1.
5. How many clock cycles does a program of 10 instructions take on the DX8, without jumps?
   $10 \cdot 8 = 80$.

04

Summary

Summary

- The DX8 runs every instruction in 8 steps: two bytes are fetched into R0 and R1, decoded into the instruction and argument registers, executed by the ALU, and written back; then the IP moves on.
- Results last only if written back: an ALU instruction writes into the register in $b_{10:8}$.
- Programs start at address 0, use R2 to R7, and end with a jump to itself.
- To program the DX8: reset on, type each byte with the data and address switches and the write button, reset off.

Before the last lecture

- Read Section 6.6.4 and the Epilogue of the book.
- Put your DX8 together in Logisim-evolution, type in the 3×4 program, and check that regOUT shows 12.
- Write a program of your own: for example, the larger of two numbers, or $1 + 2 + \cdots + 10$.

> **Next lecture** (Unit 23)
>
> Course review, and the exam.